

Go memory, GC и pprof

lifetime → escape analysis → allocations → GC → memory pressure → pprof

Основная лекция • Go 1.27.x

**Цель: научиться строить причинно-следственную цепочку от кода до CPU / RSS /
p99**

Главная цепочка лекции

Четыре упрощения, которые должны исчезнуть

значение
в коде

lifetime

escape
analysis

stack /
heap

allocations

heap
growth

GC

CPU /
memory
pressure

pprof

конкретна
я строка

Лекция строится вокруг **lifetime**: где данные живут, кто их удерживает и какую работу это заставляет выполнять **compiler/runtime**.

«указатель значит
heap»

«GC съел память»

«маленькая pause =
дешевый GC»

«RSS растёт = leak»

Вместо ярлыка — механизм → метрика → профиль → конкретный code path.

Backend живет не в абстрактной памяти

```
func handler(w http.ResponseWriter, r
*http.Request) {
    user := User{ID: 42, Name: "Alice"}
    data, _ := json.Marshal(user)
    w.Write(data)
}
```

- goroutine stacks
- heap objects
- runtime metadata
- OS thread stacks
- network buffers
- memory mappings

CPU = 95%

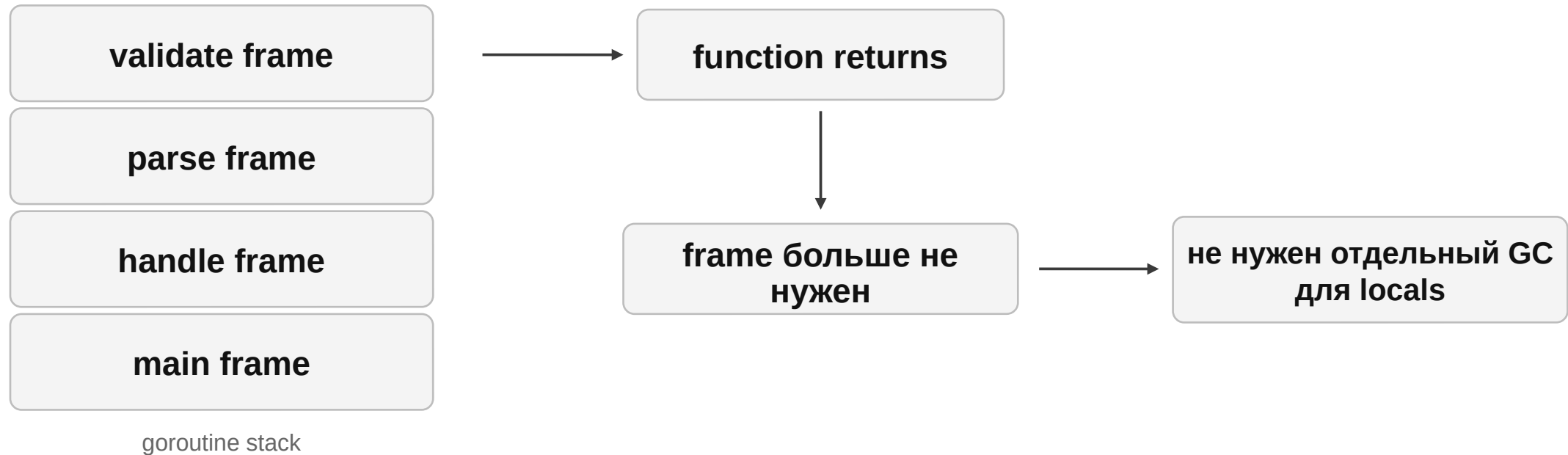
RSS = 1.8 GB

p99 = 900 ms

goroutines = 40 000

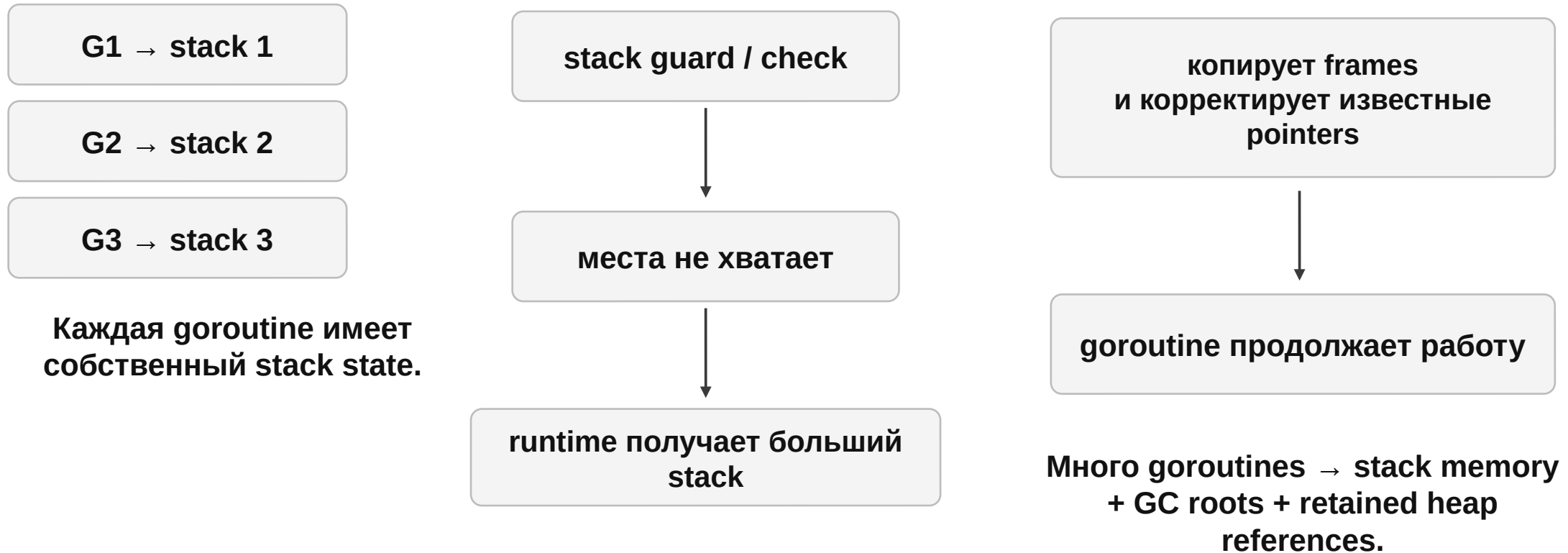
Синтаксис handler не объясняет production-симптомы. Нужна модель lifetime данных и того, как runtime управляет памятью.

Stack: lifetime следует call structure



Дешевая сторона stack — структурированный lifetime: память освобождается вместе с frame.

Goroutine stack: свой, растущий, перемещаемый



Heap: lifetime определяется reachability

Полезная модель

```
func createUser() User {  
    u := User{ID: 42}  
    return &u  
}
```

frame createUser
завершен

caller все еще держит
pointer

storage должен
пережить frame

stack
lifetime следует call structure

heap
lifetime определяется reachability

Stack / heap — решение compiler, а не синтаксис

```
func f() int {  
    p := new(int)  
    *p = 42  
    return *p  
}
```

new(T) ≠ гарантированный heap

```
func g() int {  
    x := 42  
    p := &x  
    return *p  
}
```

&x ≠ heap

```
func counter() func() int {  
    n := 0  
    return func() int { n+  
        +; return n }  
}
```

**локальная variable может жить
дольше frame**

Главный вопрос: как долго value должна существовать и может ли compiler доказать короткий lifetime?

Escape analysis: анализ потока references



- return pointer
- closure capture
- store into longer-lived object
- goroutine / независимый lifetime
- unknown/indirect call behavior
- large object/compiler constraints

correctness > потенциальная stack optimization

Если compiler не может доказать короткий lifetime, он выбирает безопасный storage strategy.

Escape analysis: неочевидные cases

goroutine
может пережить caller

returned slice
backing storage
переживает frame

interface / dynamic call
может скрыть lifetime

большой local object
может уйти в heap по
implementation reasons

Inlining меняет контекст анализа

до inlining
caller видит call boundary

body вставлен в caller

compiler доказывает
больше
→ allocations могут
исчезнуть

Allocation count — характеристика кода + toolchain version.

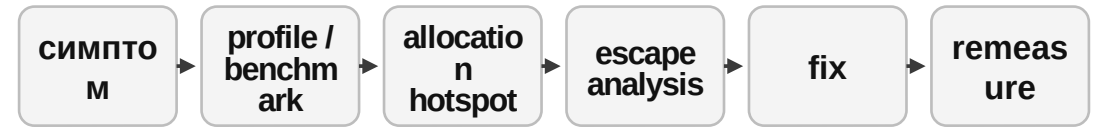
Escape analysis: как использовать в расследовании

```
go build -gcflags="-m=2" ./...
```

x escapes to heap

moved to heap: x

Но compiler diagnostics не отвечают: «это bottleneck?»



-m объясняет storage decision.
-benchmark показывает, что осталось
после optimizations.
pprof показывает, где это стало дорого.

Allocations: три разные оси

allocation count
сколько heap allocation
events

allocated bytes
сколько traffic прошло
через allocator

live bytes
сколько memory остается
reachable

Они могут радикально расходиться

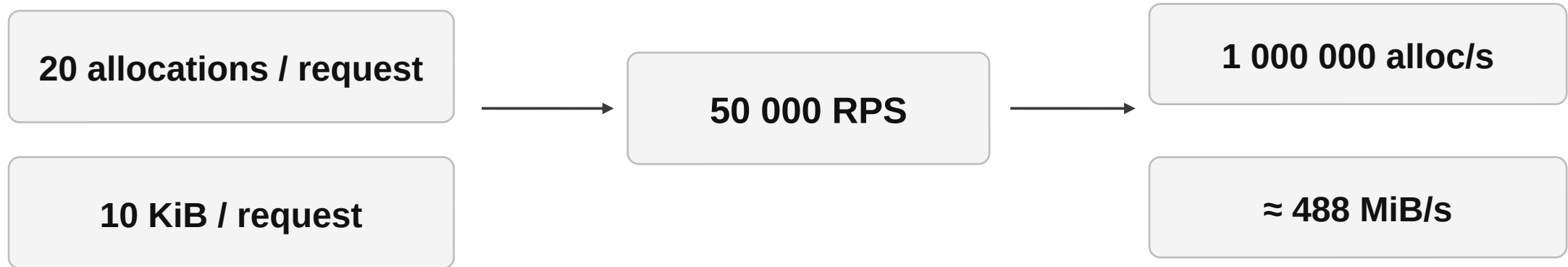
live heap = 200 MB

allocation rate = 5 GB/s

RSS $\approx$ stable

Маленький live heap может сосуществовать с огромным churn и высоким GC CPU.

Allocation rate масштабируется вместе с RPS



Performance engineering начинается с frequency × cost, а не с «эта allocation маленькая».

Откуда появляются allocations

make / backing arrays

slice, map, dynamic storage

append growth

new array + copy

string ↔ []byte

storage + copy в ряде paths

fmt / serialization

temporary buffers / interfaces

closures

captured environment

JSON

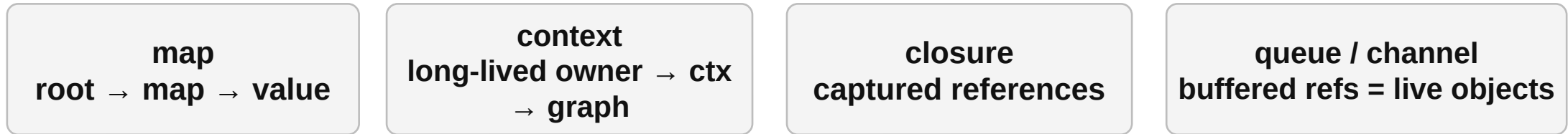
DTO + buffers + copies

Нельзя учить «construct X всегда allocates». Конкретный call path проверяем benchmark + escape diagnostics.

Ownership и retention важнее самого факта allocation



Другие owners



Иногда дополнительная compact-copy allocation уменьшает memory pressure, потому что освобождает огромный backing storage.

Pointers: lifetime, aliasing и locality

```
a := &User{Name: "Alice"}  
b := a  
b.Name = "Bob"
```

Два pointer values
→ **один storage**
→ **aliasing / shared mutable state**

[]Point

```
[X Y][X Y][X Y][X  
Y]
```

contiguous values → locality

[]*Point

```
[p][p][p][p]  
|  |  |  |  
?  ?  ?  ?
```

indirection + GC edges + cache misses

Pointer может уменьшить сору, но добавляет aliasing, lifetime extension и потенциально ухудшает locality.

Heap bytes $\neq$ GC work

Что смотреть

```
buf := make([]byte, 100<<20)
```

```
nodes := make([]*Node, n)
```

100 MiB raw bytes
почти нет outgoing Go pointers



100 MiB pointer-rich backing array
миллионы потенциальных edges

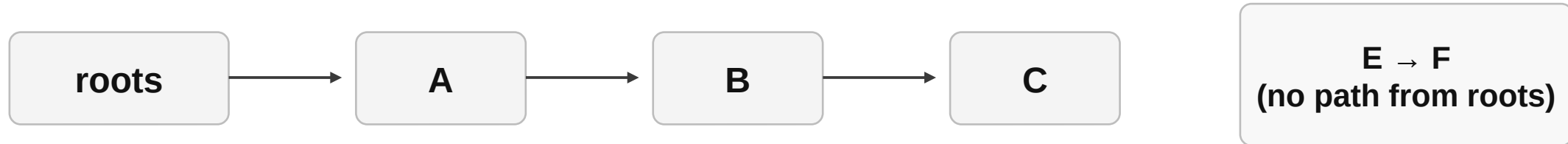
Одинаковые bytes $\rightarrow$ разная
scan work.

`/gc/scan/heap:bytes`

`/gc/scan/stack:bytes`

**objects + bytes + allocation
rate**

Garbage Collector: reachability, не бизнес-смысл



reachable → live

unreachable → collectible

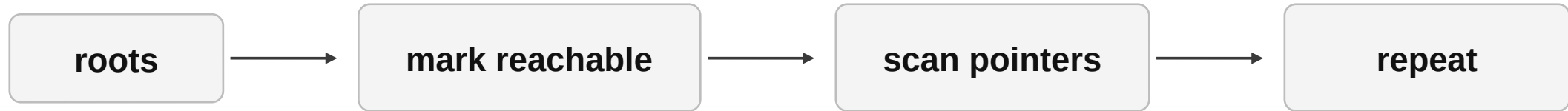
```
var cache = map[string][]byte{}  
cache[id] = hugeData
```



global root → map → entry → []byte
GC обязан сохранить

Tracing GC: mark, sweep и tri-color

Mark



Sweep



Tri-color — логическая модель progress: white → grey → black. Не физическое поле Color в каждом object.

Какой GC используется в Go

tracing

идем от roots по graph

precise

runtime знает, где Go pointers

concurrent

GC и application перекрываются во времени

parallel

несколько GC workers одновременно

mark-and-sweep

mark live → sweep reusable

non-generational

нет базовой young/old heap architecture

non-compacting

live objects обычно не уплотняются перемещением

Trade-off Go: короткие STW + concurrent progress ценой постоянного CPU/runtime overhead и fragmentation.

Concurrent GC: почему нужен write barrier

Решение

collector уже
просканировал A



A.child = C



heap graph изменился
во время tracing

pointer write



GC bookkeeping
(write barrier)



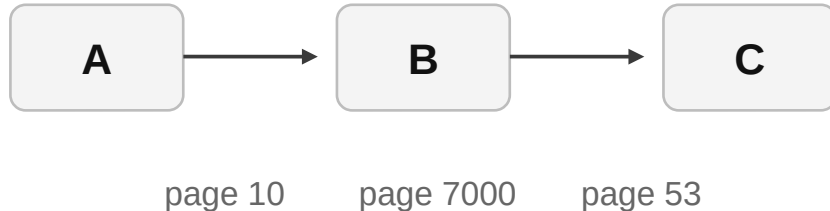
store выполняется
и collector не теряет reachability

Concurrent GC снижает длинные pauses, но добавляет background CPU, barriers, assists и cache/memory pressure.

Green Tea: оптимизируем locality marking work

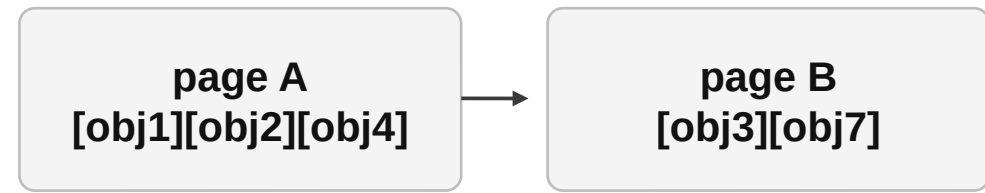
Default collector implementation с Go 1.26; programming model reachability не меняется

Старый graph flood



pointer chasing → cache misses

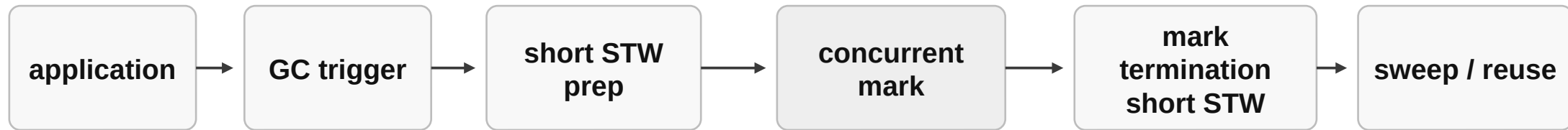
Green Tea



page-oriented scheduling
→ batching
→ better cache locality
→ smaller worklists

Green Tea уменьшает cost mark/scanning на многих workloads, но не исправляет высокий allocation rate, leaks или queue backlog.

GC cycle: это timeline, а не одна pause



Concurrent mark включает GC workers, stack scanning, write barriers и mutator assists.



GOGC: относительный memory ↔ CPU knob

Учебная модель:

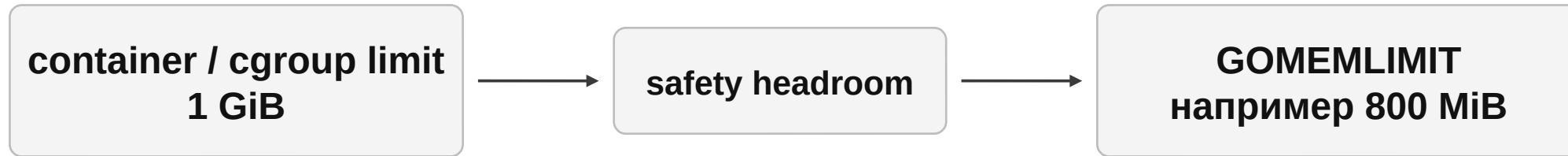
$$\text{goal} \approx \text{live heap} + \text{live heap} \times \frac{\text{GOGC}}{100}$$

Современный runtime также учитывает roots / pacer / memory limit.

GOGC	50	100	200
heap growth budget	меньше	default	больше
GC frequency	выше	средняя	ниже
GC CPU	обычно выше	baseline	обычно ниже
memory	меньше	baseline	больше

GOGC не уменьшает allocations. Он меняет, сколько memory runtime позволяет накопить между collections.

GOMEMLIMIT: soft runtime budget, cgroup: hard boundary



Почему soft?

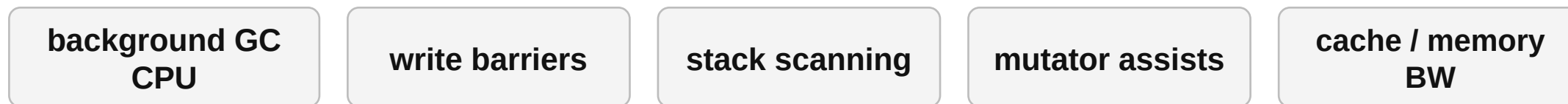


Headroom нужен под runtime overhead, stacks, native/cgo, mmap, fragmentation и bursts.

GC pauses: маленькая pause не означает дешевый GC

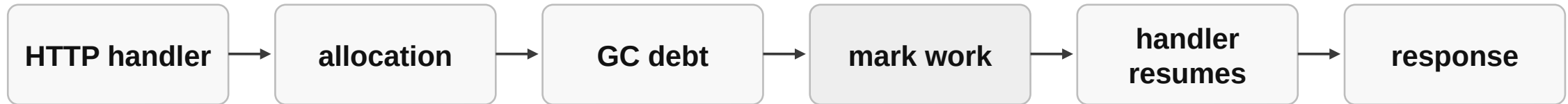


Но total cost может сидеть здесь



Для production смотрим GC frequency + GC CPU + allocation rate + live/goal + assists + p99.

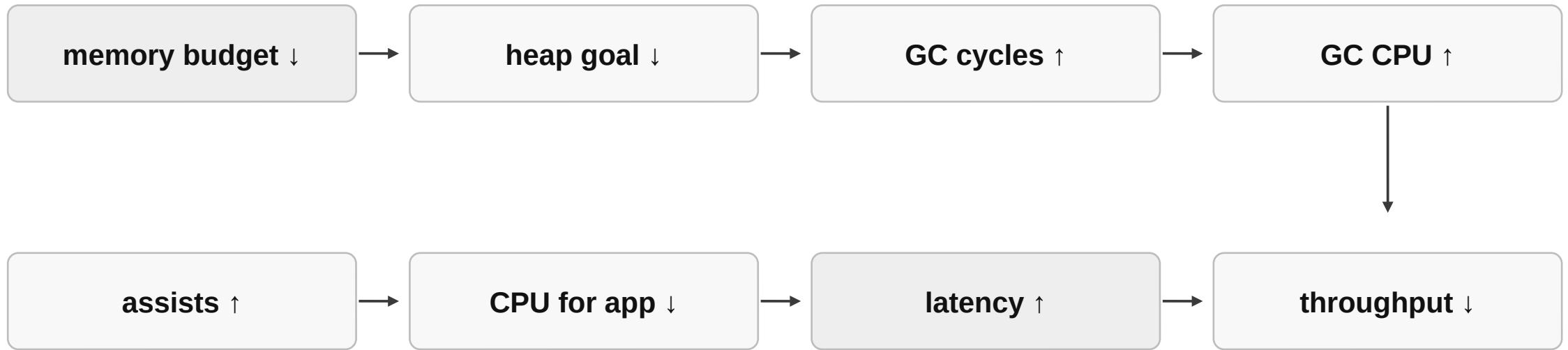
Mutator assists: GC work попадает внутрь request path



Background workers не успевают → allocating goroutine помогает collector → request wall time растёт.



Memory pressure: memory problem превращается в CPU problem



Memory pressure $\neq$ leak. Причина может быть churn, retention, goroutine growth, queue backlog или просто слишком tight budget.

Latency сама увеличивает memory: Little's Law

$$\text{in-flight} \approx \text{arrival rate} \times \text{time in system}$$



Если один request удерживает 50 KiB:

$200 \times 50 \text{ KiB} \approx 10 \text{ MiB}$

$5\,000 \times 50 \text{ KiB} \approx 244 \text{ MiB}$

DB/network slowdown может увеличить live heap без изменения кода и allocs/op.

Heap $\neq$ RSS $\neq$ virtual address space

Go runtime view

- heap objects
- heap free
- heap released
- stacks
- runtime metadata

OS / process view

- virtual address space
- RSS
- page mappings
- page faults

container view

- cgroup accounting
- hard memory limit
- host pressure

HeapAlloc может упасть, а RSS остаться высоким: память уже free для Go, но physical pages еще не reclaimed OS.

Memory leak в языке с GC: reachability живет слишком долго

Во всех случаях GC исправен

unbounded cache

slice держит большой
backing array

queue backlog

goroutine leak



Memory leak здесь — lifecycle / ownership / backpressure problem.

pprof: выбираем profile по вопросу

Симптом	Первый инструмент	Что ищем
CPU высокий	CPU profile	где реально тратятся CPU samples
live heap растёт	heap / inuse_space	кто удерживает live bytes
GC CPU высокий, heap stable	CPU + alloc_space + metrics	кто создает allocation pressure
goroutines растут	goroutine profile	где и на чем блокируются
p99 высокий, CPU низкий	trace + dependency metrics	DB/network/queue/pool waits
HeapAlloc упал, RSS нет	runtime metrics + OS/cgroup	free/released/external memory

pprof CPU: flat, cum и call path

Главная граница

go tool pprof -

<http://localhost:6060/debug/pprof/profile?seconds=30>

flat
CPU внутри
функции

cum
функция + subtree

top -cum
кто приводит к
дорогой работе

list
конкретные source
lines

p99 = 5s
CPU = 20%



DB / network / pool / blocking

**Wait ≠ CPU. Медленный endpoint
может почти не появиться в CPU
profile.**

pprof heap: churn и retention — разные вопросы

Sample type	Вопрос	Типовая задача
inuse_space	кто удерживает live bytes?	retention / leak / large live heap
inuse_objects	кто удерживает live objects?	миллионы объектов / GC work
alloc_space	кто исторически выделил bytes?	allocation churn / GC pressure
alloc_objects	кто породил objects?	tiny short-lived object storms

Один snapshot не доказывает leak. Для retention полезны $t0 \rightarrow t1 \rightarrow t2$ и diff profiles.

pprof goroutines: count важен вместе со stack traces

Сильные сигналы

goroutines = 20 000



**норма?
leak?**



смотрим dominant stacks

15 000 goroutines
→ same stack
→ waiting on channel
receive

leak candidate

10 000 goroutines
→ database/sql
→ waiting for
connection

**pool / DB pressure — root cause
надо расследовать**

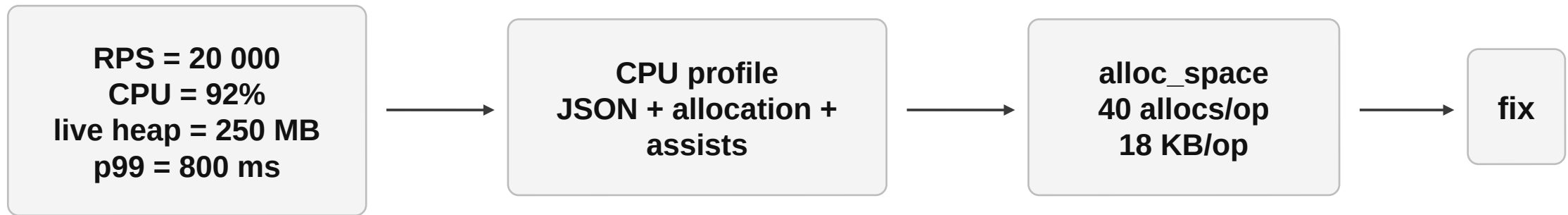
**Go 1.27: goroutineleak
автоматически находит
только определенный класс
permanent leaks**

runtime/metrics: приборная панель вокруг pprof

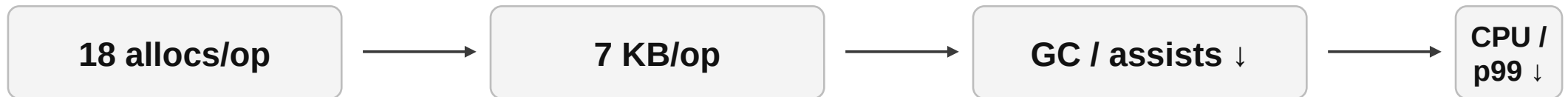
Heap state	Scan work	GC CPU	Memory classes
<code>/gc/heap/live:bytes</code> <code>/gc/heap/goal:bytes</code> <code>/gc/heap/allocs:bytes</code> <code>/gc/heap/frees:bytes</code>	<code>/gc/scan/heap:bytes</code> <code>/gc/scan/stack:bytes</code>	<code>mark/assist</code> <code>mark/dedicated</code> <code>mark/idle</code> <code>gc/total</code>	<code>heap/objects</code> <code>heap/free</code> <code>heap/released</code>

Metrics дают динамику и контекст. pprof связывает cost с code paths. Trace нужен, когда важна последовательность событий.

Production-кейс: allocation churn

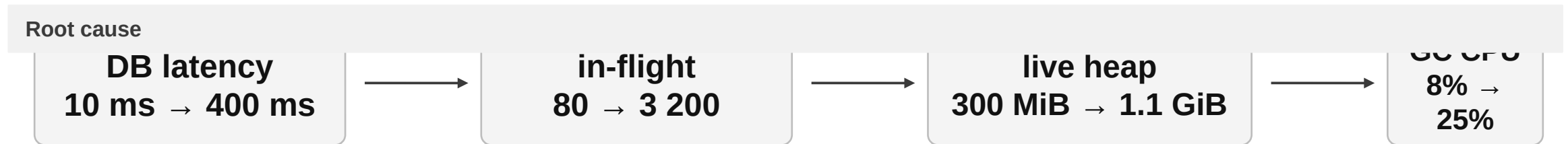


После исправления



GC bottleneck часто начинается с allocation pattern приложения, а не с настройки collector.

Production-кейс: dependency slowdown → memory pressure

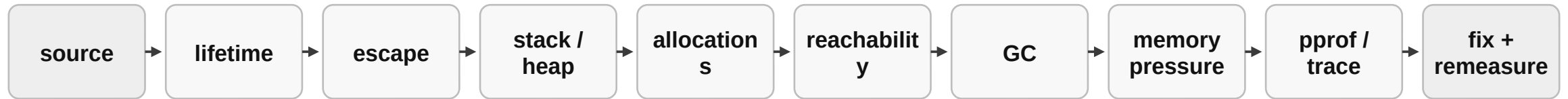


Все request objects корректно reachable. GC ничего не «теряет» и не «ломается».

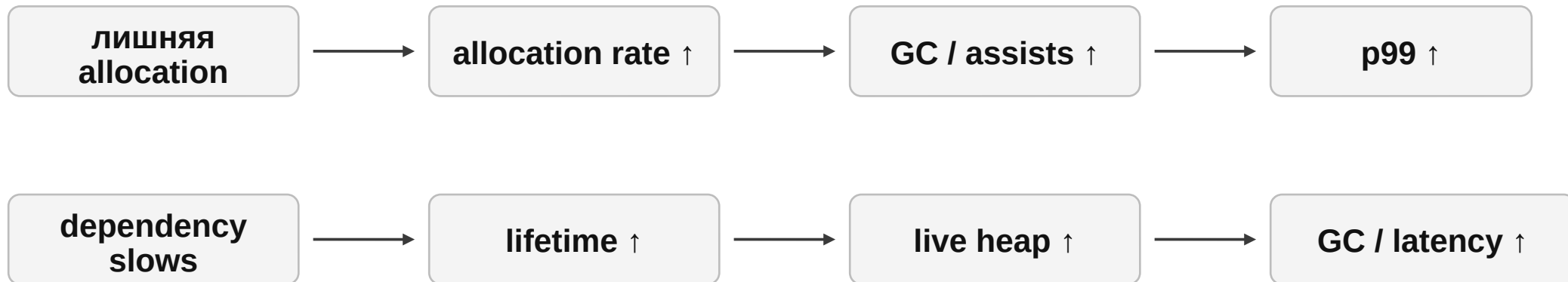
dependency latency / insufficient backpressure

Исправлять: DB / pool / timeouts / concurrency limits / load shedding / queue bounds — не первым делом GOGC=400.

Финальная схема: от source code до production diagnosis



Две production-цепочки



Главная инженерная мысль: memory performance начинается с lifetime и allocation patterns, а заканчивается измерением конкретного code path.